

44. MutationObserver

Ek built-in object jo kisi DOM element ko **observe** karta hai aur **badlav (change)** hone par callback chalata hai.

Syntax

```
js

let observer = new MutationObserver(callback);
observer.observe(node, config);
```

`config` options

- `childList`: `node` ke direct children me badlav
- `subtree`: saare descendants me badlav
- `attributes`: `node` ke attributes
- `attributeFilter`: sirf chune hue attribute names ki array
- `characterData`: `node.data` (text content)
- `attributeOldValue`: purani value bhi bhejo (`attributes` chahiye)
- `characterDataOldValue`: purana `data` bhi bhejo (`characterData` chahiye)

Callback ko `MutationRecord` objects ki list milti hai (pehla argument) aur observer khud (doosra).

`MutationRecord` properties

- `type`: `"attributes"`, `"characterData"` ya `"childList"`
- `target`: jahan badlav hua
- `addedNodes / removedNodes`: jo nodes add/remove hue
- `previousSibling / nextSibling`
- `attributeName / attributeNameSpace`
- `oldValue`: purani value (agar option on ho)

```
js
```

```
let observer = new MutationObserver(mutationRecords => {
  console.log(mutationRecords);
});
observer.observe(elem, {
  childList: true,
  subtree: true,
  characterDataOldValue: true
});
```

Ek edit me kai records bhi aa sakte hain.

Kaam kahan aata hai?

1. Integration (third-party script se): Kisi third-party script ne unwanted cheez (jaise ad `<div class="ads">`) daal di aur hatane ka tarika nahi diya. Observer se pata lagao ki wo element aaya, aur hata do. Ya jab wo kuch add kare to apna page adapt karo.

2. Architecture (ek jagah logic rakhna): Programming site par code snippets ko **Prism.js** se highlight karna hai. Dynamically load hue articles me snippets `innerHTML` se aate hain, aur har jagah (articles, quizzes, forum) `Prism.highlightElement` likhna jhanjhat hai (aur third-party module ho to patch karna mushkil). Observer se ek jagah sab handle:

js

```
let observer = new MutationObserver(mutations => {
  for (let mutation of mutations) {
    for (let node of mutation.addedNodes) {
      if (!(node instanceof HTMLElement)) continue; // text nodes skip

      if (node.matches('pre[class*="language-"]')) {
        Prism.highlightElement(node);
      }
      for (let elem of node.querySelectorAll('pre[class*="language-"]')) {
        Prism.highlightElement(elem);
      }
    }
  }
});
observer.observe(demoElem, { childList: true, subtree: true });
```

Ab jab bhi snippet aaye, apne aap highlight ho jaayega.

Extra methods

- `observer.disconnect()`: observation band karo.

- `observer.takeRecords()`: abhi tak **process na hue** records lo. `disconnect` se **pehle** call karo agar recent changes miss nahi karne (in records ke liye callback nahi chalega).

Observers node ko **weak reference** se pakadte hain, isliye observe hone ke bawajood node garbage collect ho sakta hai.

Summary

DOM me attributes, text aur elements add/remove ke badlav pakadta hai. Apne code ke badlav track karne ya third-party scripts ke saath integrate karne me kaam aata hai. `config` options sirf optimization ke liye hain (bekaar callbacks se bachne ke liye).